

HW3 Extra Credit

4.1 Shamir Secret Sharing and Error-Correcting Codes

A. The vector of n Shamir shares of a degree- $(k-1)$ polynomial is a codeword of a Reed-Solomon code because they operate in the same way. Shamir's sharing scheme is essentially an evaluation-based Reed-Solomon code, but for cryptography. For Shamir, the secret, s , is embedded in the constant term $P(0)$ of a randomly chosen polynomial of degree at most $k-1$. The shares are then distinct evaluations $P(x_1), P(x_2)$, etc, at non-zero elements. This directly relates to coding theory: evaluating a polynomial of degree $< k$ at n distinct points defines an $[n, k]_q$ Reed-Solomon code. The key difference is that Shamir uses the curve to split trust and enforce privacy, whereas Reed-Solomon uses redundant points to fix damage and corruption. Then, if someone submits bad shares, you can run Reed-Solomon to detect liars and recover the secret.

B. When shareholders submit incorrect shares, we go from erasure decoding to error correction. A Reed-Solomon code is a maximum Distance Separable code, which has a minimum Hamming distance of $d = n - k + 1$. In standard decoding, a code can correct up to $(d-1)/2$ errors. As a result, the scheme can tolerate up to $e \leq [(n - k)/2]$ corrupted shares while still guaranteeing the exact recovery of the polynomial and its secret.

C. Two domains where the equivalence between polynomial sharing and Reed-Solomon codes is vital are Byzantine Multi-Party Computation (MPC) and Fault-Tolerant Distributed Storage systems. For MPC, protocols such as BGW rely on error-correcting decoding to achieve active security against their adversaries. By running the Berlekamp–Welch algorithm during intermediate multi-party communication phases, honest parties can filter out adversarial evaluations without aborting the computation, provided fewer than $n/3$ or $n/4$ nodes are compromised. Then, for storage systems, cloud and decentralized storage networks combine privacy-preserving threshold access with erasure and error resiliency. As a result of treating Shamir secret shares as Reed-Solomon code blocks, distributed storage architectures can simultaneously protect data confidentiality across untrusted storage nodes and self-heal against silent data corruption, packet loss, and node compromise.

4.2 Byzantine Fault Tolerance

A. There are two ways in which a node can fail in distributed systems. Crash fault, which is when the node simply stops working or disconnects, or Byzantine fault, where the node behaves maliciously. This might be by sending conflicting messages to different peers, forgetting data, selectively staying silent, or colluding with other bad nodes to subvert the system. Byzantine fault tolerance itself is the property of a distributed consensus protocol that guarantees the system

continues to operate correctly. It makes sure that it continues to agree on the same sequence of valid state transactions even if a subset of the nodes show erratic behavior.

At least $3f+1$ replicas are needed because of two constraints: liveness and safety. For liveness, we note that out of n total nodes, up to f could be faulty. The network is partially synchronous, which makes a delayed message look identical to a crashed one. As a result, the system can't afford to wait for responses from all n nodes, or else a single node could stall the system forever. To guarantee progress, an operation must be able to proceed once the quorum of $Q = n-f$ responses is collected. The next portion is the constraint. To avoid waiting on up to f unresponsive nodes, the system must make decisions using a quorum size of $n-f$. In the worst case, all f Byzantine nodes eagerly participate in that quorum while f honest nodes are delayed, leaving only $n - 2f$ honest responses. To guarantee that the honest nodes cannot be outvoted or misled, their count must strictly exceed the number of Byzantine nodes ($n - 2f > f$), which requires $n \geq 3f + 1$.

B. In BFT protocols like PBFT, cryptographic primitives provide message integrity, authenticity, and non-repudiation. Cryptographic hash functions compress arbitrary-sized requests into fixed digests to ensure payload immutability, while Message Authentication Codes (MACs) and digital signatures authenticate the sender and verify that messages have not been altered in transit. Without message authentication, a Byzantine node could impersonate honest replicas, change client requests, tamper with sequence numbers and vote digests in flight, or falsely claim that other replicas committed to conflicting proposals. Furthermore, digital signatures prevent equivocation during leader changes because a malicious node cannot deny having signed a message or forge an honest replica's vote to assemble a fraudulent view-change certificate.

C. Proof-of-Work (PoW) adapts the core objective of classical BFT, therefore reaching consensus on an ordered sequence of state transitions despite malicious behavior in a permissionless, open-membership environment. While classical BFT requires a static, known roster of identified replicas where voting is counted per node and can tolerate strictly less than one-third of Byzantine nodes, PoW allows dynamic, anonymous participation where influence is tied to computational hash rate rather than node identity, tolerating up to strictly less than one-half of adversarial mining power under synchronous network assumptions. Moreover, classical BFT provides immediate, deterministic finality as soon as a $2f + 1$ quorum commits a block, whereas PoW provides probabilistic (Nakamoto) finality, meaning a transaction's settlement certainty increases exponentially with block depth but always retains a negligible, non-zero risk of reorganization.